

ST7 : High Performance Computing
Virtual Seismic Imaging with SEM3D
Earthquake Simulation and Underground Exploring

Ngoc Long Tran, Grégoire Delignon, Giovanni Benedetti, Nicolò Dal Monte

April 13, 2026

Contents

1	Introduction	3
1.1	Context and Objective	3
1.2	Main Contributions	3
2	Theoretical Background	5
2.1	Elastic Wave Propagation	5
2.2	Spectral Element Method (SEM)	5
2.3	Inverse Problem and Misfit Functional	5
2.4	Adjoint-State Method	6
2.5	Reverse Time Migration Workflow	6
3	Methodology and Implementation	7
3.1	Project Workflow	7
3.2	HPC Execution	9
3.2.1	Parallelization Strategy	9
3.2.2	Motivation for the Design Choice	9
3.2.3	Slurm Configuration for Gradient and Search Direction	9
3.2.4	Mesher and Solver Configuration	10
3.2.5	<code>sbatch_and_wait()</code> : Slurm Job Orchestration Utility	10
3.3	Preprocessing and Forward Simulation	11
3.4	Misfit Computation	12
3.5	Adjoint Computation	12
3.6	Practical Implementation and Gradient Computation	13
3.6.1	Gradient Mapping to Material Mesh and Parallelization	13
3.7	Compute Search Direction	14
3.7.1	The L-BFGS Algorithm	14
3.7.2	Parallelization Strategy	15
3.7.3	Global Vector Reconstruction	15
3.8	Backtracking Line Search and Material Update	16
4	Experimental Setup and Results	18
4.1	Experimental Configuration	18
4.2	Module Verification	18
5	Discussion	19
5.1	What Worked	19
5.2	Technical Limitations: The Material Configuration Challenge	19

6	Conclusion	20
6.1	Summary	20
6.2	Future Work	20

Chapter 1

Introduction

1.1 Context and Objective

Before starting any construction project, the question of choosing the location is among the most important, in which numerous factors are examined. For a nuclear plant, the stability and stiffness of the ground are primordial criteria but understanding the Earth's properties is not a question easy to answer.

In this project, we are using the high performance computing to simulate the propagation of seismic waves through a domain and building its geophysical map by exploiting data that are recorded at the surface level.

1.2 Main Contributions

In this project, we implemented a Full-Waveform Inversion (FWI) workflow to identify an unknown subsurface composition. Following the methodology proposed by **Fathi, Kallivokas, and Poursartip** in "*Full-waveform inversion in three-dimensional PML-truncated elastic media*" ([1]), we utilized the **SEM3D** solver to execute high-performance seismic simulations.

Our key contributions include:

- **Algorithm Implementation:** We translated the FWI mathematical framework into a functional pipeline, specifically handling elastic wave propagation within **PML-truncated** boundaries to simulate an infinite medium without artificial boundary reflections.
- **Subsurface Discovery:** Starting from a "blind" scenario where the domain properties were not provided, we successfully reconstructed the material composition and geological structure using only recorded sensor signals.
- **FWI Workflow Development:** Execution of the complete iterative inversion cycle, consisting of:
 - **Forward Modeling:** Generating synthetic seismograms with SEM3D.
 - **Misfit Computation:** Calculating the difference between synthetic and observed data.
 - **Adjoint-State Method:** Generating adjoint sources and computing the gradient kernels to update the model.

- **HPC/MPI Execution:** Leveraging parallel computing via MPI to manage the significant memory and processing demands of 3D elastic wave inversion, a core requirement of the ST7 High Performance Computing framework.

Chapter 2

Theoretical Background

2.1 Elastic Wave Propagation

The physical foundation of this study is the propagation of elastic waves within a three-dimensional, semi-infinite, and heterogenous medium [1]. We characterize the mechanical behavior of the soil using the displacement field $\mathbf{u}(\mathbf{x}, t)$ and the material's constitutive properties defined by the Lamé parameters, λ and μ [3]. Within the interior domain Ω^{RD} , the governing partial differential equation (PDE) for elastodynamics is given by:

$$\text{div}\{\mu[\nabla\mathbf{u} + (\nabla\mathbf{u})^T] + \lambda(\text{div } \mathbf{u})\mathcal{I}\} + \mathbf{b} = \rho\ddot{\mathbf{u}} \quad (2.1)$$

where ρ denotes the mass density, $\mathcal{I}$ is the second-order identity tensor, and $\mathbf{b}$ represents the body force per unit volume [1].

2.2 Spectral Element Method (SEM)

To simulate a semi-infinite medium on a finite computational grid, we employ **Perfectly-Matched-Layers (PMLs)** at the truncation boundaries to absorb outgoing waves and eliminate artificial reflections [1]. SEM3D solves these equations using a spectral element discretization [2]. Unlike traditional Finite Element Methods, SEM utilizes high-degree Lagrange polynomials at **Gauss-Lobatto-Legendre (GLL)** nodes. This choice results in a strictly diagonal mass matrix M , which significantly reduces the computational overhead for time-stepping.

The resulting semi-discrete system is:

$$M\ddot{\mathbf{d}}^{st} + C\dot{\mathbf{d}}^{st} + K(m)\mathbf{d}^{st} + G\bar{\mathbf{d}}^{st} = \mathbf{f}(t) \quad (2.2)$$

where $K(m)$ is the stiffness matrix dependent on the material parameters $m = [\lambda, \mu]$, and C, G represent the damping and coupling matrices for the PML buffer [2]. This system is integrated using an explicit **fourth-order Runge-Kutta (RK-4)** scheme, allowing for massive parallelization via MPI in HPC environments.

2.3 Inverse Problem and Misfit Functional

The objective of the inverse problem is to identify the spatial distribution of the Lamé parameters that best explains the recorded surface data $\mathbf{d}_{obs}$. We formulate this as a

PDE-constrained optimization problem where we minimize a misfit functional $\mathcal{J}(m)$ [4]:

$$\mathcal{J}(m) = \frac{1}{2} \int_0^T \sum_{j=1}^{N_r} \|\mathbf{u}(\mathbf{x}_j, t; m) - \mathbf{d}_{obs}(\mathbf{x}_j, t)\|^2 dt + \mathcal{R}(m) \quad (2.3)$$

Here, N_r is the number of receivers and $\mathcal{R}(m)$ is a regularization term used to stabilize the inversion. Following Fathi et al. [1], we utilize regularization to handle the "blind" nature of the domain and mitigate the ill-posedness of the problem.

2.4 Adjoint-State Method

To compute the gradient of the misfit functional $\nabla \mathcal{J}$ with respect to the material parameters, we utilize the **Adjoint-State Method**. This approach avoids the direct computation of the Jacobian matrix. By introducing an adjoint field $\mathbf{w}$, the sensitivity kernels (gradients) for λ and μ are computed via the zero-lag cross-correlation of the forward field $\mathbf{u}$ and the time-reversed adjoint field $\mathbf{w}$ [3]:

$$\frac{\partial \mathcal{J}}{\partial \lambda} = - \int_0^T (\text{div } \mathbf{u})(\text{div } \mathbf{w}) dt, \quad \frac{\partial \mathcal{J}}{\partial \mu} = - \int_0^T \nabla \mathbf{u} : \nabla \mathbf{w} dt \quad (2.4)$$

2.5 Reverse Time Migration Workflow

The characterization of the underground composition is performed through an iterative workflow [4, 1]:

1. **Forward Modeling:** Simulate wave propagation in the current trial model using SEM3D to obtain synthetic seismograms.
2. **Data Residuals:** Compute the difference between synthetic traces and observed signals at receiver locations.
3. **Adjoint Source Construction:** The residuals are time-reversed and injected as adjoint sources at the receiver positions.
4. **Adjoint Simulation:** Solve the adjoint wave equation to back-propagate the error signals into the domain.
5. **Kernel Interaction:** Interact the forward and adjoint wavefields to calculate the local sensitivity of the parameters.
6. **Update:** Update the model parameters (λ, μ) using a gradient-based optimization algorithm and repeat until the misfit converges.

Chapter 3

Methodology and Implementation

3.1 Project Workflow

The overall workflow of the implemented inversion procedure is summarized in Algorithm 1. Starting from an initial material model, the method iteratively alternates between forward simulation, misfit evaluation, adjoint computation, gradient-based search direction estimation, and model update through a backtracking line search.

Algorithm 1 Main inversion workflow

```
1: Input:
2:   Initial material model
3:   Observed traces
4:   Receiver positions

5: Preprocessing:
6:   Run the mesher

7: while  $J > \text{tol}$  do
8:   Forward simulation
9:     Run the forward SEM3D solver
10:    Generate simulated traces
11:   Misfit computation
12:     Compute the residual  $r(t) = u_{\text{sim}} - u_{\text{obs}}$ 
13:     Compute the misfit  $J$ 
14:   Adjoint computation
15:     Reverse the residuals in time
16:     Write one source file per receiver and component
17:     Generate the adjoint input file from a template
18:     Run the adjoint SEM3D solver to back-propagate the residuals
19:   Gradient computation
20:     Launch the dedicated post-processing step
21:     Compute the global gradients from forward and adjoint fields
22:   Search direction computation
23:     Compute the L-BFGS search directions
24:     Load the global search directions
25:   Backtracking line search
26:     Initialize  $\alpha_\lambda, \alpha_\mu$ 
27:   while the Armijo condition is not satisfied do
28:     Update model, run forward simulation, recompute misfit
29:     Reduce step lengths
30:   end while
31:   Material update
32:     Accept the step lengths found by line search
33:     Update the material model
34:     Synchronize forward and adjoint configurations
35: end while

36: Output:
37:   Updated Lamé parameter fields
```

This algorithm reflects the practical structure of the implemented main script, where each inversion iteration combines a forward simulation, an adjoint simulation, a dedicated MPI-based gradient/L-BFGS post-processing stage, and a backtracking line search requiring additional forward evaluations.

3.2 HPC Execution

The computational cost of Full-Waveform Inversion (FWI) is extremely high, as each iteration requires multiple large-scale simulations and data processing steps. To address this challenge, we leveraged High Performance Computing (HPC) resources and parallelization strategies at different levels of the workflow.

3.2.1 Parallelization Strategy

Instead of initializing a single MPI environment in the main script, we adopted a modular approach where parallelism is activated only in the computationally intensive steps. In particular, MPI is used in:

- the forward and adjoint simulations (SEM3D solver),
- the gradient computation,
- the search direction computation (L-BFGS).

The mesher and solver components are already implemented with MPI parallelization within SEM3D. Therefore, they can be directly executed in parallel without additional modifications.

For the gradient and search direction computation, we developed a dedicated MPI-based script, in which each rank processes a subset of the domain and collaborates through collective communication operations.

3.2.2 Motivation for the Design Choice

This design avoids creating a global MPI environment at the main level, which would remain idle during sequential operations such as file generation, data I/O, and other operations like job submission and iteration management. In these phases, no parallel computation is performed, and maintaining active MPI processes would lead to unnecessary resource occupation and suboptimal utilization of the HPC system.

This approach provides several advantages:

- **Improved resource utilization:** computational resources are allocated only during parallel sections.
- **Reduced overhead:** no need to maintain an active MPI communicator throughout the entire workflow.
- **Modularity:** each step can be developed, tested, and executed independently.
- **Scalability:** different steps can be run with different parallel configurations depending on their computational requirements.

3.2.3 Slurm Configuration for Gradient and Search Direction

The gradient and search direction computation is executed through a Slurm job using multiple nodes and MPI processes. The most relevant configuration parameters are:

- `#SBATCH -nodes=4`: number of compute nodes allocated to the job.
- `#SBATCH -ntasks=4`: number of MPI processes (one per node in this case).
- `#SBATCH -cpus-per-task=32`: number of CPU cores per MPI process, enabling hybrid parallelism.

This configuration allows each MPI rank to handle a portion of the computational domain while exploiting multi-core parallelism locally. The combination of distributed (MPI) and shared-memory (multi-threading) parallelism ensures efficient utilization of the available HPC resources.

3.2.4 Mesher and Solver Configuration

The mesher and solver are executed through Slurm jobs with configurations adapted to their computational requirements.

The mesher is a relatively lightweight preprocessing step and is executed on a single node:

- `#SBATCH -nodes=1`
- `#SBATCH -ntasks=1`
- `#SBATCH -cpus-per-task=32`

In contrast, the solver is the most computationally intensive component and is executed in parallel across multiple nodes:

- `#SBATCH -nodes=8`
- `#SBATCH -ntasks=128`
- `#SBATCH -ntasks-per-node=16`

This configuration enables distributed wavefield simulation using MPI, while maintaining a balanced workload across nodes.

3.2.5 `sbatch_and_wait()`: Slurm Job Orchestration Utility

To simplify the execution of Slurm batch scripts on the HPC cluster, we implemented a helper function named `sbatch_and_wait()`. This function takes as input the path of a batch script, submits it using the `sbatch` command, and waits until the job has finished before continuing the execution of the Python workflow.

The goal of this function is to automate job management and enforce a sequential execution of the inversion pipeline. In our workflow, several steps must be executed in a strict order: for example, the forward simulation must complete before computing the misfit, and the adjoint simulation must complete before evaluating the gradient. The function ensures this synchronization between the Python script and the Slurm scheduler.

After submitting the job, the function extracts the job ID from the `sbatch` output and periodically checks its status using `squeue`. Once the job is no longer present in the queue, the function queries `sacct` to retrieve its final state.

This approach offers several advantages. It removes the need for manually checking job status with Slurm commands, and avoids submitting each step individually. It also provides useful feedback, such as the job ID, execution time, and final state. Finally, it improves robustness: if a job does not complete successfully, the function raises an error and stops the workflow, preventing the use of incomplete results.

Overall, `sbatch_and_wait()` provides a simple and effective way to control the execution of HPC jobs from a Python script.

3.3 Preprocessing and Forward Simulation

Before entering the actual algorithm, a preprocessing stage is required to prepare the mesh and the simulation setup.

The mesher is responsible for generating the computational mesh of the domain and partitioning it into subdomains for parallel execution. In our case, it is driven by the file `mesh.input`, where the number of partitions is set to 128. In this mode, the mesher also relies on auxiliary files describing the geometry and material layering of the domain.

The mesher produces the partitioned mesh files `mesh4spec.*.h5`; the generated mesh files must then be placed inside the `sem/` directory, which is the location expected by the solver.

Beyond mesh generation, preprocessing in our workflow also includes loading the receiver positions from `stations.txt` and copying the mesh directory into the adjoint configuration folder. In our implementation, we chose to maintain two separate directories for the forward and adjoint simulations. This design allows us to manage independently the configuration files and outputs of the two problems. For this reason, the mesh directory (`sem/`) produced during the meshing step is duplicated into the adjoint configuration folder. This ensures that both forward and adjoint simulations are run on the same discretized domain.

Once the mesh has been prepared, the forward problem is solved using `sem3d`. Unlike the mesher, the solver is driven by the configuration file `input.spec`, which defines all the simulation parameters. These include the simulation duration, the mesh reference through `mesh_file = "mesh4spec"`, the material description through `mat_file = "material.spec"`, the source definition, the receiver configuration (via `stations.txt`), the time integration scheme, and the output variables to be saved during the run.

In our implementation, we use `material.spec` instead of a static material file. This choice allows the solver to read material properties from external `.h5` files, which are updated at each iteration.

Therefore, the forward solver takes as input the partitioned mesh generated by the mesher, the material configuration (linked to the HDF5 files), the receiver positions, and the source specification defined in `input.spec`.

The forward simulation produces two main categories of output. First, it generates receiver time histories, stored in the `traces/` directory (in HDF5 format in our case). Second, it produces volumetric wavefield data, such as snapshots of the displacement, velocity, and strain fields, stored in the `res/` directory.

Both the mesher and the solver are launched the `sbatch_and_wait()` function described previously.

3.4 Misfit Computation

After each forward simulation, the synthetic traces produced by SEM3D are compared with the observed traces recorded at the receiver positions. This step corresponds to the evaluation of the misfit functional, which measures the discrepancy between simulated and observed data. In our case, we use the classical time-domain L^2 distance between the two sets of displacement signals, consistently with the formulation introduced in the literature.

More precisely, if $u_{\text{sim}}(x_r, t)$ denotes the synthetic displacement at receiver r and $u_{\text{obs}}(x_r, t)$ the corresponding observed signal, the discrete misfit is computed as

$$J = \frac{1}{2} \sum_n \sum_r \|u_{\text{sim}}(x_r, t_n) - u_{\text{obs}}(x_r, t_n)\|^2 \Delta t, \quad (3.1)$$

which is the discrete counterpart of the continuous L^2 misfit used in the theoretical formulation.

In practice, this step is implemented in the function `compute_misfit()`, which reads both simulated and observed traces from their respective HDF5 folders using `ParseSEM3DH5Traces`. Only the displacement field is used, and all three spatial components ($\mathbf{x}$, $\mathbf{y}$, $\mathbf{z}$) are extracted for each receiver.

A practical difficulty is that simulated and observed traces do not necessarily share the same time discretization. For this reason, the observed traces are first interpolated onto the simulated time grid using a linear interpolation. Once both datasets are expressed on the same temporal grid, the residual is computed as

$$r(t) = u_{\text{sim}}(t) - u_{\text{obs}}^{\text{interp}}(t). \quad (3.2)$$

The final misfit is then evaluated as

$$J = \frac{1}{2} \sum r^2 \Delta t_{\text{sim}}, \quad (3.3)$$

where Δt_{sim} is the time step of the simulated traces. Besides the scalar value J , the function also returns the residual tensor, the simulated time vector, and the simulated time step.

3.5 Adjoint Computation

Once the misfit has been evaluated, the residual is used to define the sources of the adjoint problem. The adjoint simulation is obtained by injecting the data residuals at the receiver locations, after reversing them in time. This is exactly the procedure described in the literature: the original physical source is replaced by a set of misfit sources located at the receivers and flipped from $t = T$ to $t = 0$. Moreover, one source must be defined for each receiver and for each spatial component.

In our implementation, the residual tensor returned by the misfit function has shape $(N_t, 3, N_r)$, where N_t is the number of time samples, 3 is the number of displacement components, and N_r is the number of receivers. The first operation therefore consists in reversing the residual along the time axis. This produces the time-reversed residual.

The time-reversed residual is then written to disk through the function `write_time_reversed_residual_files()`. This function generates one text file per receiver and per component, following the convention `misfit_{receiver}_{component}.txt`.

Each file contains two columns: the simulation time vector and the corresponding source amplitude. As a result, for N_r receivers, a total of $3N_r$ adjoint source files are produced.

After these files have been created, the function `write_backward_spec_from_template()` automatically generates the configuration file for the adjoint run. Starting from a template `input.spec`, the function inserts one `source` block for each receiver and each direction. Each block specifies the receiver coordinates, the source type, the unit direction vector ($\mathbf{x}$, $\mathbf{y}$, or $\mathbf{z}$), and the corresponding time file.

Once the adjoint configuration has been generated, the adjoint simulation is launched through the dedicated Slurm batch script. As in the forward step, the execution is handled through `sbatch_and_wait()`, which guarantees that the adjoint job finishes before the workflow proceeds to the gradient computation. The adjoint problem is solved with SEM3D on the same mesh as the forward simulation, but with different sources: the physical source is replaced by the time-reversed residuals. The resulting adjoint wavefield represents the backward propagation of the data mismatch inside the domain and will be used in the next step to evaluate the gradients of the Lamé parameters.

3.6 Practical Implementation and Gradient Computation

In the practical implementation using SEM3D, the gradient computation is derived from the first variation of the Lagrangian functional. As specified in the project framework, this involves the interaction of five distinct physical and mathematical quantities: the forward displacement ($\mathbf{u}$) and PML stress ($\mathbf{S}$), their adjoint counterparts ($\mathbf{w}$ and $\mathbf{T}$), and the material parameters (m).

The domain is discretized into hexahedral elements, which are further subdivided into cells using **Gauss-Lobatto-Legendre (GLL)** quadrature points. Each element contains $(N + 1)^3$ GLL nodes (typically 125 nodes for $N = 4$). A key practical feature is that these GLL points are concentrated at the corners and edges of the elements, allowing for seamless displacement continuity and gradient accumulation across element boundaries.

The computation follows a two-equation system:

1. **The State Equation:** Resolves the forward wavefield ($\mathbf{u}, \mathbf{S}$).
2. **The Adjoint Equation:** Resolves the backward wavefield ($\mathbf{w}, \mathbf{T}$) using the time-reversed residuals as sources at sensor locations.

At every GLL node, the gradient is accumulated by correlating the strain of the forward field with the adjoint field. Because the GLL nodes coincide with the integration points, the resulting mass matrix is diagonal, which allows the five quantities to be computed efficiently in a parallel MPI environment without the need for global matrix inversions.

3.6.1 Gradient Mapping to Material Mesh and Parallelization

Once the sensitivity kernels are computed at each GLL quadrature point, the resulting gradients must be mapped onto the nodal positions of the material mesh to facilitate the model update. This step is necessary because the material parameterization is often defined on a separate discretization that may not perfectly align with the spectral element quadrature points used by the solver.

To handle this mapping efficiently, we implement a spatial search algorithm. Specifically, we utilize a **KD-Tree** structure (via a standard Python library such as **SciPy**) to identify the closest GLL point for each node in the material mesh. This nearest-neighbor approach allows for the direct assignment of the computed sensitivity values to the corresponding nodes of the material model.

To maintain the efficiency and scalability of the parallel pipeline, this mapping process is distributed across the MPI processors. Rather than performing a global search, each rank identifies and assigns the material nodes that fall within its local physical subdomain. By assigning each node to the correct processor based on the initial domain partitioning, we ensure that the program remains fully parallelized. This avoids centralized data bottlenecks and ensures that the material update—and the subsequent generation of updated HDF5 files—is handled consistently by the specific ranks responsible for each portion of the domain.

3.7 Compute Search Direction

3.7.1 The L-BFGS Algorithm

Standard gradient descent methods can exhibit slow convergence in complex valleys, while full Newton methods require the computation and inversion of the Hessian matrix. In 3D seismic imaging with unstructured meshes, the parameter space N (number of mesh elements) is large. Storing a dense $N \times N$ Hessian matrix is memory-intensive and computationally expensive.

The L-BFGS algorithm approximates the inverse Hessian matrix using the history of the last M optimization iterations. We maintain two FIFO queues of size M :

- $s_k = m_k - m_{k-1}$: the difference in the model parameters (the physical step).
- $y_k = g_k - g_{k-1}$: the difference in the gradient vectors (the change in curvature).

The core of the algorithm is the *two-loop recursion*. It applies a sequence of vector dot products and scalar multiplications to transform the current gradient g_k into a search direction r_k , pointing towards the minimum of the misfit function. Assuming an initial inverse Hessian approximation of the identity matrix, the algorithm follows this logic:

Algorithm: L-BFGS Two-Loop Recursion

Input: Current gradient g , queues s and y of size M , MPI communicator $comm$

Output: Search direction vector r

```

q = g
// First loop (backward pass)
For i = M-1 down to 0:
    alpha[i] = global_dot(s[i], q) / global_dot(y[i], s[i])
    q = q - alpha[i] * y[i]

r = q
// Second loop (forward pass)
For i = 0 up to M-1:
    beta = global_dot(y[i], r) / global_dot(y[i], s[i])

```

```

    r = r + s[i] * (alpha[i] - beta)

return -r

```

3.7.2 Parallelization Strategy

The MPI environment is initialized during the preceding gradient computation step. Consequently, the search direction function does not initialize MPI from scratch, instead, it receives the `comm` as an input parameter. This allows an easy transition between evaluating the gradient and computing the search direction without the overhead of rebuilding the parallel environment.

Within this active parallel environment, the data remains partitioned. Each MPI rank holds a specific subset of the mesh (local chunks). Most operations in the L-BFGS two-loop recursion, such as vector additions and scalar multiplications, are performed element-wise locally.

However, the algorithm relies on scalar products (e.g., `dot(y, s)`) to evaluate the curvature of the misfit function. A local dot product represents only a fraction of the domain. To address this, we utilized the `MPI_Allreduce` operation: each rank computes its local dot product using `numpy.dot`, and the `Allreduce` (with `MPI.SUM`) aggregates these values across the network, returning the global scalar to all processes simultaneously. This allows every rank to scale its local descent direction properly without holding the full model in memory.

3.7.3 Global Vector Reconstruction

Once the local search directions (r_λ, r_μ) are computed, the sequential main script requires the global arrays to perform the Backtracking Line Search and update the HDF5 physical files.

Reconstructing the global vector from distributed chunks requires preserving the physical ordering of the nodes. Because the unstructured mesh is partitioned dynamically to optimize physical domains, a direct concatenation via `MPI_Gatherv` would alter this ordering, leading to incorrect model updates.

To reconstruct the global arrays, we exploited the NumPy’s advanced indexing and the original mapping indices (`mat_global_indices`):

1. We compute the total size of the global mesh (N_{tot}) using an `MPI_Allreduce` with the `MPI.MAX` operator on the local mapping indices.
2. Every rank allocates an empty array of zeros of size N_{tot} .
3. Each rank places its local computed chunk at the correct physical indices using the local map: `local_full[mat_global_indices] = local_chunk`.
4. We perform an `MPI_Reduce` with `MPI.SUM` to Rank 0. Since the spatial partitions are mutually exclusive across ranks, the zero-masks overlap correctly.

This method fuses the distributed pieces into an ordered, global array on the master rank. Rank 0 then saves these vectors (`dir_lam, dir_mu`) along with the global scalar products in a compressed `.npz` file. The sequential main script loads this file and update the model during the Line Search, keeping the workflow decoupled.

3.8 Backtracking Line Search and Material Update

The final step of the inversion iteration is the update of the model parameters. Once the search direction is computed via the L-BFGS algorithm, a backtracking line search is employed to find the optimal step length. This process ensures that the updated model leads to a reduction in the misfit function by satisfying the Armijo condition.

In this implementation, the material update involves modifying the Lamé parameters, λ and μ . These updates are written into HDF5 files. The workflow is designed so that the solver can read these updated values for the subsequent forward simulation. This transition is managed by switching the solver's configuration from "material.input" to "material.spec" in order to use external material specifications, allowing for a dynamic evolution of the velocity model across iterations.

The backtracking line search is designed to use the directions provided by L-BFGS to find new distributions that significantly decrease the cost function (here, the misfit J). It is described in [1] (p. 15). With a ρ parameter equal to 0.5, our practical implementation turns out to be the Algorithm 2 on the page below.

Some remarks on the inputs of the backtracking line search:

1. The scalar products between the directions and the gradients p_λ and p_μ are calculated in parallel: indeed, in each processor, we can compute the scalar product between the local versions of the gradient and direction, and then sum the contributions of all the local scalar products to obtain the global ones (p_λ and p_μ).
2. Please also note that we manipulate two different formats: HDF5 files, which use 3D NumPy grids to store the Lamé parameters distributions on the "material" mesh, and 1D vectors to store the directions d_λ and d_μ . To be able to update the 3D grids with the 1D vectors, we therefore need a "mapping list" that maps each index of the direction vectors d_λ and d_μ to the 3 coordinates (x,y,z) of the corresponding point in the 3D grids. This list is built just before the gradients computation, and comes from the spatial search algorithm described in section 3.6.1. This allows us to use the function "modify h5" which will identify that point within the 3D grids of the λ and μ distributions.

Moreover, for practical reasons, at the end of this algorithm we also update the h5 files that are used in the simulation of the adjoint problems.

Algorithm 2 Backtracking line search

```
1: Input:
2:   Search directions  $d_\lambda$  and  $d_\mu$ 
3:   Current misfit  $J$ 
4:   Scalar products  $p_\lambda$  and  $p_\mu$ 
5:   Mapping list

6: Initialization:
7:    $J_{learn} = +\infty$ 
8:    $J_{thresh} = -\infty$ 
9:    $V_\mu = 2\alpha_\mu d_\mu$ 
10:   $V_\lambda = 2\alpha_\lambda d_\lambda$ 

11: while  $J_{learn} > J_{thresh}$  do
12:   Test a new distribution of  $\lambda$  and  $\mu$ 
13:      $V_\mu = V_\mu - \alpha_\mu d_\mu$ 
14:      $V_\lambda = V_\lambda - \alpha_\lambda d_\lambda$ 
15:   Call a function that modifies the h5 files
16:     modify h5( $V_\mu$ , mapping list)
17:     modify h5( $V_\lambda$ , mapping list)
18:   Launch SEM3D to simulate the forward problem with the new  $\lambda$  and  $\mu$ 
   distributions
19:     sbatch and wait(SOLVER.sbatch)
20:   Retrieve the traces calculated by the solver, and compute the misfit
   associated to these new distributions
21:      $J_{learn} = \text{compute misfit}(\text{traces}, \text{obs u})$ 
22:   Compare it to the threshold from Armijo's condition
23:      $J_{thresh} = J + c_1(\alpha_\lambda p_\lambda + \alpha_\mu p_\mu)$ 
24:   Modify the step lengths in case Armijo's condition is not satisfied by
   the distributions we have just tested
25:      $\alpha_\lambda = 0.5\alpha_\lambda$ 
26:      $\alpha_\mu = 0.5\alpha_\mu$ 
27: end while

28: Output:
29:   Updated Lamé parameter fields
```

Chapter 4

Experimental Setup and Results

4.1 Experimental Configuration

The numerical experiments are conducted on a 3D elastic domain. To manage the computational intensity of the Full-Waveform Inversion, the domain is partitioned into 128 subdomains for parallel processing.

- **Resources:** The simulations are executed on 8 high-performance computing nodes, utilizing 128 MPI tasks.
- **Synchronization:** The execution of the forward solver, adjoint solver, and post-processing scripts is orchestrated using a custom utility that manages Slurm job dependencies, ensuring each stage completes before the next begins.
- **Boundary Conditions:** PML (Perfectly Matched Layers) are applied to simulate a semi-infinite medium and avoid artificial reflections.

4.2 Module Verification

Although the complete automated loop encountered integration issues, the individual components of the pipeline were successfully verified:

- **Forward and Adjoint Simulations:** The solver successfully produced synthetic wavefields and back-propagated the residuals to generate adjoint fields.
- **Gradient Computation:** The MPI-based scripts correctly performed the cross-correlation of forward and adjoint wavefields to produce sensitivity kernels.
- **Optimization Logic:** The L-BFGS recursion was validated by successfully computing search directions from the history of gradients and model updates.

—

Chapter 5

Discussion

5.1 What Worked

The project successfully established a modular High-Performance Computing (HPC) framework for seismic inversion. The strategy of activating MPI environments only for computationally intensive tasks—specifically the SEM3D solver and the L-BFGS gradient processing—proved efficient for resource utilization. The custom orchestration utility, `sbatch_and_wait()`, successfully managed job dependencies, ensuring that the forward and adjoint simulations were fully synchronized before any post-processing began. Furthermore, the MPI-based gradient computation and the L-BFGS two-loop recursion were validated, demonstrating that the distributed data chunks could be correctly fused into a global search direction.

5.2 Technical Limitations: The Material Configuration Challenge

The primary obstacle encountered was the transition between material definition formats required for the iterative inversion loop. While the initial forward simulation functioned correctly using a static `material.input` file, the Full-Waveform Inversion (FWI) workflow requires the solver to read updated Lamé parameters from external HDF5 files via the `material.spec` configuration.

We successfully managed to have every individual step work separately: the solver executed correctly, gradients were computed, and the search direction was determined. However, the system failed when attempting to switch the solver from `material.input` to `material.spec` for the second iteration. We have identified that the domain must be built from the very beginning using the HDF5 material files for the mapping to remain consistent across iterations. Because the initial mesh was not built with these specific HDF5 references, a mismatch occurred between the partitioned mesh files (`mesh4spec.*.h5`) and the material property updates. This configuration error prevented the solver from correctly mapping the updated parameters back to the spectral elements, thereby halting the automated inversion loop.

Chapter 6

Conclusion

6.1 Summary

This project established a high-performance computing framework for 3D Full-Waveform Inversion. We successfully implemented the necessary building blocks: forward modeling, adjoint-state gradient computation, and a parallelized L-BFGS optimization scheme. The workflow effectively leverages MPI and Slurm to handle the heavy computational load of 3D elastic wave simulations.

6.2 Future Work

To achieve a fully operational inversion loop, future work will focus on harmonizing the initial mesh generation with the material update scripts. Specifically, the domain setup must be modified to accept HDF5 material files from the first step. Once this link is established, the workflow will be capable of performing multi-iteration inversions to reconstruct complex subsurface structures.

Bibliography

- [1] Arash Fathi, Loukas F Kallivokas, and Babak Poursartip. Full-waveform inversion in three-dimensional pml-truncated elastic media. *arXiv preprint arXiv:1504.08340*, 2015.
- [2] Filippo Gatti, Sara Touhami, Fernando Lopez-Caballero, and Didier Clouteau. SEM3D: A high-performance numerical earthquake simulator. *LMPS/CentraleSupélec*, 2018. Developed by LMPS, CNRS, CEA, and IPGP.
- [3] Albert Tarantola. *Inverse problem theory and methods for model parameter estimation*. SIAM, 2005.
- [4] Jean Virieux and Stephane Operto. An overview of full-waveform inversion in exploration geophysics. *Geophysics*, 74(6):WCC1–WCC26, 2009.